COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	CPE21S1
DATE PERFORMED:	Aug 4, 2026	**DATE SUBMITTED:**	**Aug 12, 2026**
NAME:	Yazinel Corro	**INSTRUCTOR:**	Engr.JimLord Quejado

ACTIVITY NO. 5.1 Seatwork 5.1: Queue

1. PROCEDURE OUTPUT

ILO A

:

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> c );

int main(){
    std::queue<char> q;

    q.push('c');
    q.push('p');
    q.push('e');
    q.push('0');
    q.push('0');
    q.push('0');

    //display
    display(q);
    //empty
    std::cout<<"is the queue empty? "<<q.empty() <<"\n";

    //size
    std::cout<<"size of queue: "<<q.size() <<"\n";

    //front
    std::cout<<"front of queue: " <<q.front() <<"\n";

    //back
    std::cout<<"back of queue: " <<q.back() <<"\n";

    //pop
    std::cout << "pop another element: ";
    q.pop();

    //display
    display(q);

    // push another element
    std::cout<<"push another element: ";
```

```
    //display
    display(q);

    // push another element
    std::cout<<"push another element: ";
    q.push('a');
    q.push('b');
    q.push('c');
    std::cout<<"\n";

    //display
    std::cout<<"display element: ";
    display(q);

    //delete the queue
    while (!q.empty()) {
        q.pop();
    }

    //display after deleting the queue
    std::cout << "queue after deleting: ";
    display(q);

    //check if the queue is empty
    std::cout << "is the queue empty? " << q.empty() << "\n";

}

void display(std::queue<char> r){
    //create a copy of the queue
    std::queue<char> c = r;

    while(!c.empty()){
        std::cout<<" "<<c.front();
        c.pop();
    }
    std::cout<<"\n";
}
```

Output of ILOA:

```
 c p e 0 0 0
is the queue empty? 0
size of queue: 6
front of queue: c
back of queue: 0
pop another element:  p e 0 0 0
push another element:
display element:  p e 0 0 0 a b c
queue after deleting:
is the queue empty? 1
PS C:\Users\TIPQC\OneDrive\Documents\CPE 010 - CPE21S1\HOA5p1\ILOA>
```

Analysis:

- The ILOA shows the queue list. It shows the size, front, back, pop, push. Display, and after q after deleting. The syntax that I put is q.size, q.front, q.empty, q.back, and etc. The syntax for these is similar so the ILOA is a little bit simple and I understand each of it.

ILOB 1:

Main. cpp

HOA5p1 > ILOB > main.cpp > main()

```cpp
#include <iostream>
#include "queuell.h"


int main() {


   qNode<char> *front =nullptr;
   qNode<char> *back = nullptr;

   enqueue(&front, &back, 'J');
   std::cout << front->data << " " << back->data << std::endl;

   enqueue(&front, &back, 'I');
   std::cout << front->data << " " << back->data << std::endl;

   enqueue(&front, &back, 'M');
   std::cout << front->data << " " << back->data << std::endl;

   dequeue(&front, &back);
   std::cout << front->data << " " << back->data << std::endl;

    std::cout << "Full Queue List: \n"; display(front);
    std::cout << " Front Variable: \n"<< Front(front)<<std::endl;
   return 0;
}


```

Header file:

```
#ifndef QUEUELL_H



template <typename T>
class qNode {
   public:
   T data;
   qNode* next;
};


//creating a new node
template <typename T>
qNode<T>* new_node(T newData) {
   qNode<T>* newNode = new qNode<T>;
   newNode->data = newData;
   newNode->next = nullptr;
   return newNode;
}


template <typename T>
void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) {
   //create a new node
   qNode<T> *newNode = new_node<T>(newData);


   //inserting to an empty queue
   if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
       (*frontPtr) = newNode;
       (*backPtr) = newNode;
   }
```

```
#ifndef QUEUELL_H
void enqueue(qNode<T> **frontPtr, qNode<T> **backPtr, T newData) {
```

```

    //inserting to an empty queue
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
    }


    //inserting an item into a non empty queue
    (*backPtr)->next = newNode; //point the backPtr next to the newNode
    (*backPtr) = newNode;
}


template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr) {
    //create a temporary node to store the node to be deleted
    qNode<T> *deleteNode = nullptr;
    deleteNode = (*frontPtr);


    //check if the queue is only 1 node
    if ((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }
```

```
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }


    //deleting of the node
    (*frontPtr) = deleteNode->next;
    delete deleteNode;
}


//display all elements in the list

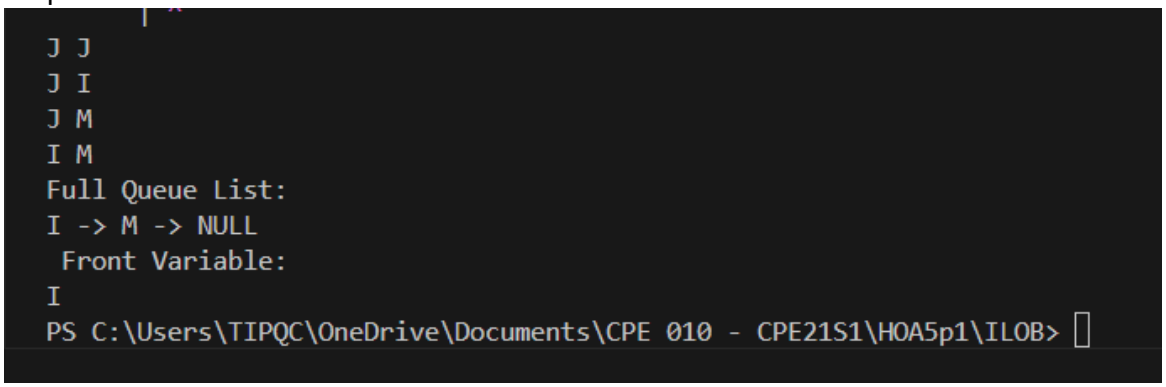

//return the front variable


#endif

```

```
64  //display all elements in the list
65  template <typename T>
66  void display (qNode<T>* frontPtr){
67      if (frontPtr == nullptr){
68          std::cout << "Queue is empty.\n";
69          return;
70      }
71      qNode<T>* current = frontPtr;
72      while (current != nullptr) {
73          std::cout << current -> data << " -> ";
74          current = current -> next;
75      }
76      std:: cout << "NULL \n";
77  }
78
79  //return the front variable
80  template <typename T>
81  T Front (qNode<T>* frontPtr) {
82      if (frontPtr != nullptr){
83          return frontPtr->data;
84      }
85  }
86
87  #endif
88
```

Output of ILOB1:



Analysis of ILOB:

-
- In this procedure, I created a queue using linked nodes and used enqueue to add the characters J, I, and M to the queue. I then used dequeue to remove the first element and observed how the front and back values changed. The display and Front functions were also used to check the contents of the queue and identify the current front element.
-

ILOB 2:

main cpp

main..cpp queueArr.h

main..cpp > ...

```cpp
#include <iostream>
#include "queueArr.h"

int main() {
    //create and object queue
    queueArr<int> q(6);
    q.Enqueue(1);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(2);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(3);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(4);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(5);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(6);
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Enqueue(7);
    q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Dequeue();
    std::cout << "q_front: " << q.Front() << " q_back: " << q.Back() << " q_size: " << q.Size() << std::endl;
    q.Clear();

    queueArr<int> q2(q);

    std::cout << "\n--- Copy Constructor ---" << std::endl;
    std::cout << "q2_front: " << q2.Front() << " q2_back: " << q2.Back() << " q2_size: " << q2.Size() << std::endl;
    queueArr<int> q3(6);

    q3 = q;
    std::cout << "\n--- Copy Assignment Operator ---" << std::endl;
    std::cout << "q3_front: " << q3.Front() << " q3_back: " << q3.Back() << " q3_size: " << q3.Size() << std::endl;
    return 0;
}

```

Header:

queueArr.h > Clear()

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//Array-Based Circular Queue
template <typename T>
class queueArr {
    private:
        //create a pointer array:
        T *q_array;
        size_t q_capacity;
        size_t q_size;
        int q_front;
        int q_back;

    public:
        //prototypes:
        queueArr(size_t capacity);
        bool Empty();
        bool Full();

        size_t Size();
        void Clear();

        T Front();
        T Back();

        void Enqueue(T newData);
        void Dequeue();

        ~queueArr();

        //tasks:
        queueArr(const queueArr &other);
        queueArr &operator = (const queueArr &other);
};

//constructor:
template <typename T>
```

```cpp
//constructor:
template <typename T>
queueArr<T>::queueArr(size_t capacity) {

    //capacity of the array
    q_capacity = capacity;

    //current size of the array
    q_size = 0;

    //initial front and back index
    q_front = 0;
    q_back = -1;

    //newly allocated array
    q_array = new T[q_capacity];
    std::cout << "Constructor class was called\n";
}

//destructor:
template <typename T>
queueArr<T>::~queueArr() {
    delete [] q_array;
};

//empty:
template <typename T>
bool queueArr<T>::Empty() {
    return (q_size == 0);
}

//full:
template <typename T>
bool queueArr<T>::Full() {
    return (q_size == q_capacity);
}
```

```
78  void queueArr<T>::Clear() {
79      q_front = 0;
80      q_back = -1;
81      q_size = 0;
82  }
83
84  //front:
85  template <typename T>
86  T queueArr<T>::Front() {
87      //check and display if the queue is empty
88      if(Empty()) {
89          std::cout << "The queue is empty\n";
90          return T();
91      }
92
93      //return the front item
94      return q_array[q_front];
95  }
96
97  //back:
98  template <typename T>
99  T queueArr<T>::Back() {
100     //check and display if the queue is empty
101     if(Empty()) {
102         std::cout << "The queue is empty\n";
103         return T();
104     }
105
106     //return the back item
107     return q_array[q_back];
108 }
109
110 //size:
111 template <typename T>
112 size_t queueArr<T>::Size() {
113     return q_size;
114 }
115
116 //enqueue:
```

```cpp
template <typename T>
void queueArr<T>::Enqueue(T newData) {
    //check if the queue if full
    if(Full()) {
        std::cout << "Queue is full" << std::endl;
        return;
    }

    //compute for the q_back of the queue array
    q_back = (q_back + 1) % q_capacity;
    // 5 > 0, 6 > 1, 7 > 2, clue:use modulo

    //assign the new value to the back of the queue array
    q_array[q_back] = newData;

    //increment the q_size
    q_size++;
}

//dequeue:
template <typename T>
void queueArr<T>::Dequeue() {
    //check if its empty
    if(Empty()) {
        std::cout << "Queue is empty" << std::endl;
        return;
    }

    //compute for the q_front to move it circulary
    q_front = (q_front + 1) % q_capacity;

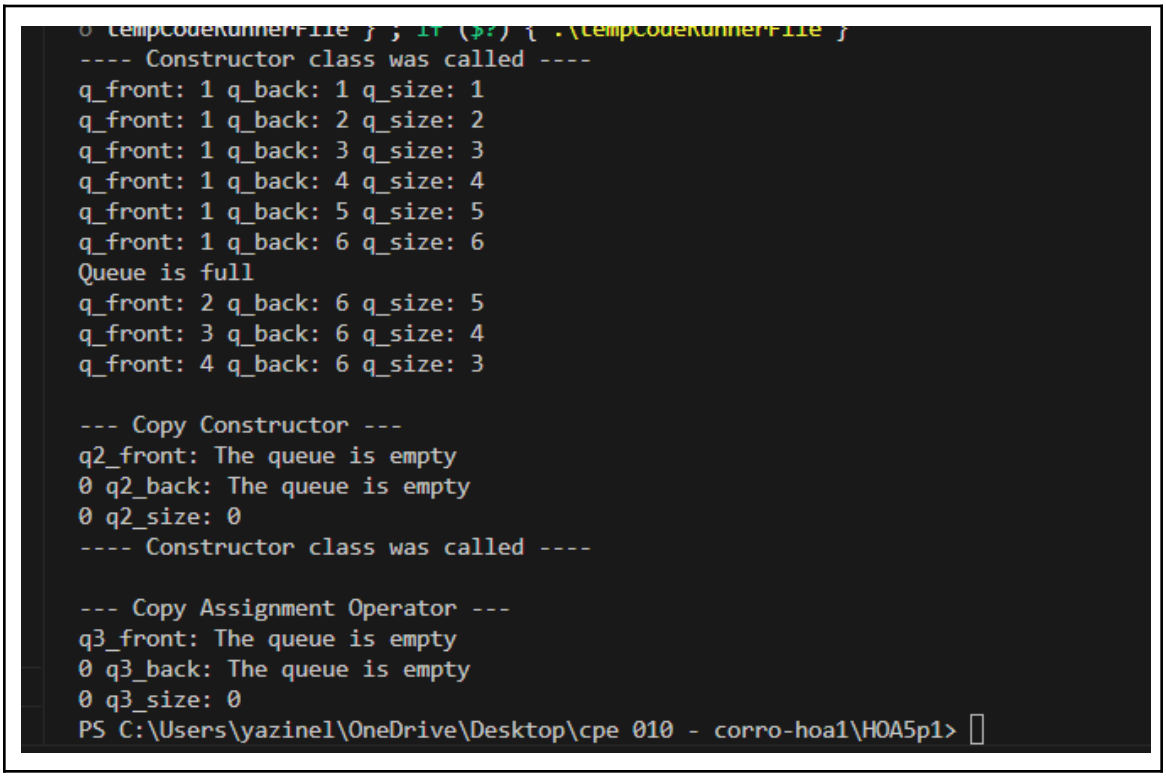
    //decrement the q_size
    q_size--;
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr &other) {
    //copy properties from the other
```

```
151
152 //copy constructor
153 template <typename T>
154 queueArr<T>::queueArr(const queueArr &other) {
155     //copy properties from the other
156     q_capacity = other.q_capacity;
157     q_size = other.q_size;
158     q_front = other.q_front;
159     q_back = other.q_back;
160
161     //allocate new memory for the array
162     q_array = new T[q_capacity];
163
164     //copy the elements from the other queue
165     for (size_t i = 0; i < q_size; i++) {
166         size_t index = (q_front + q_back) % q_capacity;
167         q_array[index] = other.q_array[index];
168     }
169 }
170
171 //copy assignment operator
172 template <typename T>
173 queueArr<T> &queueArr<T>::operator=(const queueArr<T> &other) {
174     if (this != &other) {
175         //clean up existing memory
176         delete [] q_array;
177
178         //copy properties from the other
179         q_capacity = other.q_capacity;
180         q_size = other.q_size;
181         q_front = other.q_front;
182         q_back = other.q_back;
183
184         //allocate new memory for the array
185         q_array = new T[q_capacity];
186
187         //copy the elements from the other queue
```

```
1   #ifndef QUEUEARR_H
172 template <typename T>
173 queueArr<T> &queueArr<T>::operator=(const queueArr<T> &other) {
174     if (this != &other) {
175         //clean up existing memory
176         delete [] q_array;
177
178         //copy properties from the other
179         q_capacity = other.q_capacity;
180         q_size = other.q_size;
181         q_front = other.q_front;
182         q_back = other.q_back;
183
184         //allocate new memory for the array
185         q_array = new T[q_capacity];
186
187         //copy the elements from the other queue
188         for (size_t i = 0; i < q_size; i++) {
189             size_t index = (q_front + q_back) % q_capacity;
190             q_array[index] = other.q_array[index];
191         }
192     }
193     return *this;
```

Output:



Analysis:
In the procedure, I created a circular queue with a capacity of 6 and tested adding and removing values. The output showed that the queue became full after adding 6 values, and the next value could not be added. I also observed that Dequeue removes the front value and decreases the queue size, while Clear resets the queue.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

Main.cpp

```cpp
#include <iostream>
#include "Printer.h"

int main() {

    std::cout << "-- SHARED PRINTER SIMULATION --- " << std::endl;

    Printer printer(8);

    // Create print jobs
    Job job1(101, "Flux", 10);
    Job job2(102, "Kiel", 5);
    Job job3(103, "Uzi", 20);
    Job job4(104, "Win", 8);
    Job job5(105, "Aia", 6);
    Job job6(106, "Dos", 7);
    Job job7(107, "Sam", 12);
    Job job8(108, "Xia", 25);
    std::cout << "\n--- Adding Print Jobs ---" << std::endl;

    printer.AddJob(job1);
    printer.AddJob(job2);
    printer.AddJob(job3);
    printer.AddJob(job4);
    printer.AddJob(job5);
    printer.AddJob(job6);
    printer.AddJob(job7);
    printer.AddJob(job8);

    printer.ProcessJobs();

    std::cout << "Simulation finished." << std::endl;
    return 0;
}
```

Header

```cpp
#ifndef PRINTER_H
#define PRINTER_H

#include <iostream>
#include <string>


class Job {
private:
    int jobID;
    std::string userN;
    int pages;

public:

    Job() {
        jobID = 0;
        userN = "";
        pages = 0;
    }

    Job(int id, std::string name, int numPages) {
        jobID = id;
        userN = name;
        pages = numPages;
    }

    int getJobID() {
        return jobID;
    }

    std::string getUserN() {
        return userN;
    }

    int getPages() {
        return pages;
    }

    void displayJob() {
        std::cout << "Job ID: " << jobID << std::endl;
        std::cout << "User: " << userN << std::endl;
        std::cout << "Pages: " << pages << std::endl;
    }
};
```

```
40      void displayJob() {
41          std::cout << "Job ID: " << jobID << std::endl;
42          std::cout << "User: " << userN << std::endl;
43          std::cout << "Pages: " << pages << std::endl;
44      }
45  };
46
47  template <typename T>
48  class queueArr {
49  private:
50      T* q_array;
51      size_t q_capacity;
52      size_t q_size;
53      int q_front;
54      int q_back;
55
56  public:
57
58      // Constructor
59      queueArr(size_t capacity) {
60          q_capacity = capacity;
61          q_size = 0;
62          q_front = 0;
63          q_back = -1;
64
65          q_array = new T[q_capacity];
66      }
67
68      // Destructor
69      ~queueArr() {
70          delete[] q_array;
71      }
72
73      // Empty
74      bool Empty() {
75          return q_size == 0;
76      }
77
78      // Full
79      bool Full() {
80          return q_size == q_capacity;
81      }
82
83      // Size
84      size_t Size() {
85          return q_size;
```

```
84      size_t Size() {
85          return q_size;
86      }
87
88      // Front
89      T Front() {
90          if (Empty()) {
91              std::cout << "Queue is empty." << std::endl;
92              return T();
93          }
94
95          return q_array[q_front];
96      }
97      // Back
98      T Back() {
99          if (Empty()) {
00              std::cout << "Queue is empty." << std::endl;
01              return T();
02          }
03
04          return q_array[q_back];
05      }
06      void Enqueue(T newData) {
07          if (Full()) {
08              std::cout << "Queue is full." << std::endl;
09              return;
10          }
11          q_back = (q_back + 1) % q_capacity;
12          q_array[q_back] = newData;
13          q_size++;
14      }
15
16      void Dequeue() {
17          if (Empty()) {
18              std::cout << "Queue is empty." << std::endl;
19              return;
20          }
21
22          q_front = (q_front + 1) % q_capacity;
23
24          q_size--;
25      }
26
```

```
void Dequeue() {

void Clear() {
    q_front = 0;
    q_back = -1;
    q_size = 0;
}

queueArr(const queueArr& other) {

    q_capacity = other.q_capacity;
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;

    q_array = new T[q_capacity];

    for (size_t i = 0; i < q_size; i++) {

        size_t index = (q_front + i) % q_capacity;

        q_array[index] = other.q_array[index];
    }
}

// Copy Assignment Operator
queueArr& operator=(const queueArr& other) {

    if (this != &other) {

        delete[] q_array;

        q_capacity = other.q_capacity;
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        q_array = new T[q_capacity];

        for (size_t i = 0; i < q_size; i++) {

            size_t index = (q_front + i) % q_capacity;

            q_array[index] = other.q_array[index];
```

```
166                 size_t index = (q_front + i) % q_capacity;
167
168                 q_array[index] = other.q_array[index];
169             }
170         }
171
172         return *this;
173     }
174 };
175
176 class Printer {
177 private:
178
179     queueArr<Job> jobs;
180
181 public:
182
183     Printer(int capacity) : jobs(capacity) {
184     }
185
186
187     void AddJob(Job newJob) {
188
189         if (jobs.Full()) {
190
191             std::cout << "Printer queue is full!"
192                       << std::endl;
193
194             return;
195         }
196
197         jobs.Enqueue(newJob);
198
199         std::cout << "\nJob added successfully!"
200                   << std::endl;
201
202         newJob.displayJob();
203
204         std::cout << "Jobs waiting: "
205                   << jobs.Size()
206                   << std::endl;
```

```
189             if (jobs.Full()) {
194                 return;
195             }
196
197             jobs.Enqueue(newJob);
198
199             std::cout << "\nJob added successfully!"
200                       << std::endl;
201
202             newJob.displayJob();
203
204             std::cout << "Jobs waiting: "
205                       << jobs.Size()
206                       << std::endl;
207         }
208
209
210         void ProcessJobs() {
211
212             if (jobs.Empty()) {
213
214                 std::cout << "\nNo print jobs available."
215                           << std::endl;
216
217                 return;
218             }
219
220             std::cout << "\n===== PROCESSING PRINT JOBS ====="
221                       << std::endl;
222
223
224             while (!jobs.Empty()) {
225
226
227                 Job currentJob = jobs.Front();
228
229                 std::cout << "\nPrinting job..."
230                           << std::endl;
231
232                 currentJob.displayJob();
233
234                 std::cout << "Printing "
235                           << currentJob.getPages()
```

```
                << std::endl;


        while (!jobs.Empty()) {


            Job currentJob = jobs.Front();

            std::cout << "\nPrinting job..."
                      << std::endl;

            currentJob.displayJob();

            std::cout << "Printing "
                      << currentJob.getPages()
                      << " pages..."
                      << std::endl;

            std::cout << "Job "
                      << currentJob.getJobID()
                      << " completed!"
                      << std::endl;


            jobs.Dequeue();

            std::cout << "Jobs remaining: "
                      << jobs.Size()
                      << std::endl;
        }

        std::cout << "\nAll print jobs have been completed."
                  << std::endl;
    }
};

#endif
```

Output

```
--- Adding Print Jobs ---

Job added successfully!
Job ID: 101
User: Flux
Pages: 10
Jobs waiting: 1

Job added successfully!
Job ID: 102
User: Kiel
Pages: 5
Jobs waiting: 2

Job added successfully!
Job ID: 103
User: Uzi
Pages: 20
Jobs waiting: 3

Job added successfully!
Job ID: 104
User: Win
Pages: 8
Jobs waiting: 4

Job added successfully!
Job ID: 105
User: Aia
Pages: 6
Jobs waiting: 5

Job added successfully!
Job ID: 106
User: Dos
Pages: 7
Jobs waiting: 6

Job added successfully!
Job ID: 107
User: Sam
Pages: 12
Jobs waiting: 7
```

```
Job added successfully!
Job ID: 108
User: Xia
Pages: 25
Jobs waiting: 8

===== PROCESSING PRINT JOBS =====

Printing job...
Job ID: 101
User: Flux
Pages: 10
Printing 10 pages...
Job 101 completed!
Jobs remaining: 7

Printing job...
Job ID: 102
User: Kiel
Pages: 5
Printing 5 pages...
Job 102 completed!
Jobs remaining: 6

Printing job...
Job ID: 103
User: Uzi
Pages: 20
Printing 20 pages...
Job 103 completed!
Jobs remaining: 5

Printing job...
Job ID: 104
User: Win
Pages: 8
Printing 8 pages...
Job 104 completed!
Jobs remaining: 4

Printing job...
Job ID: 105
```

```
Printing job...
Job ID: 105
User: Aia
Pages: 6
Printing 6 pages...
Job 105 completed!
Jobs remaining: 3

Printing job...
Job ID: 106
User: Dos
Pages: 7
Printing 7 pages...
Job 106 completed!
Jobs remaining: 2

Printing job...
Job ID: 107
User: Sam
Pages: 12
Printing 12 pages...
Job 107 completed!
Jobs remaining: 1

Printing job...
Job ID: 108
User: Xia
Pages: 25
Printing 25 pages...
Job 108 completed!
Jobs remaining: 0

All print jobs have been completed.
Simulation finished.
PS C:\Users\yazinel\OneDrive\Desktop\cpe 010 - corro-hoa1\HOA5p1\HOA5p1_supp>
```

CONCLUSIONS

Summary of lessons learned:

- In this activity, I learned how to implement an array-based circular queue in C++. I learned how Enqueue adds an element to the back of the queue, while Dequeue removes an element from the front. I also learned how Front, Back, Size, Empty, and Clear are used to manage and check the queue.

Analysis of the procedure:

- In the procedure, We created a queue with a capacity of 6 and added different values to it. I observed the front, back, and size after every enqueue operation, then tested what happens when the queue is full. After that, It used Dequeue to remove elements and observed how the front and size changed. I also add the copy construct and copy assignment construct. Finally, I used Clear to reset the queue.

Analysis of the supplementary activity:

- The output is the way it is because the printer follows the First In, First Out principle, so the jobs are processed in the same order they were added. Each new job is placed at the back of the queue, while the first job is removed from the front. This makes sure that the users who submitted their jobs first are served first. I also used a queue to simulate a shared printer where jobs are processed in the order they are submitted.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

- Overall, I think I did well in this activity because I was able to understand how a queue works and apply it to the printer simulation. I still need more practice with the circular queue and its operations so I can understand them better.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: code assistance

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.